

Lesson 6: CSS Basics

Intro to Programming — Code the Dream

Session Overview

By the end of this session, you will be able to:

- Explain what CSS is and how it connects to HTML
- Write CSS rules using selectors, properties, and values
- Use the box model to control spacing with padding and margin
- Create a flexible layout using Flexbox
- Add a navigation bar to a webpage

Explore

What Is CSS?

HTML gives a page its **structure**.

CSS gives a page its **style**.

```
body {  
  background-color: #f0f0f0;  
  color: #333;  
  font-family: Arial, sans-serif;  
}
```

CSS stands for **Cascading Style Sheets**.

These three lines change the background, text color, and font for the entire page.

How to Link CSS to HTML

Create a file called `index.css` inside a `css` folder.

Then link it in the `<head>` of your HTML file:

```
<head>
  <title>My Portfolio</title>
  <link rel="stylesheet" href="css/index.css">
</head>
```

The `<link>` tag tells the browser: "apply the styles in this file."

The `href` value must match the exact path to your CSS file.

CSS Syntax

Every CSS rule follows the same pattern:

```
selector {  
  property: value;  
}
```

A real example:

```
h1 {  
  color: navy;  
  font-size: 2em;  
}
```

- **Selector** — which element to style (`h1`)
- **Property** — what to change (`color`, `font-size`)
- **Value** — what to change it to (`navy`, `2em`)

Three Types of Selectors

Element — styles every instance of that tag:

```
p { color: gray; }
```

Class — styles any element with that class name:

```
.highlight { background-color: yellow; }
```

ID — styles one specific element:

```
#about { padding: 20px; }
```

The Cascade: Which Style Wins?

When two rules target the same element, CSS has to choose one.

```
p { color: blue; }           /* lower specificity */  
.red-text { color: red; } /* higher specificity */
```

```
<p class="red-text">Hello World</p>
```

The text will be **red** — the class rule wins.

This is the "Cascading" part of CSS. More specific rules override less specific ones.

Check for Understanding

**An element has both a tag rule and a class rule targeting the same property.
Which one applies?**

- A) The tag rule — it was written first
- B) The class rule — classes are more specific than tags
- C) Neither — they cancel each other out
- D) Both — the styles are combined

The Box Model

Every HTML element is a **box** with four layers:

- **Content** — the text or image inside
- **Padding** — space between content and the border
- **Border** — the visible edge of the box
- **Margin** — space outside the border, between elements

```
section {  
  padding: 20px;  
  margin-bottom: 30px;  
  border: 1px solid #ccc;  
}
```

Tip: add a colorful border to any element to see its box clearly.

Padding vs. Margin

Padding = space *inside* the box (between content and border)

Margin = space *outside* the box (between this element and its neighbors)

```
.card {  
  padding: 16px;    /* breathing room inside */  
  margin: 24px;     /* pushes neighbors away */  
}
```

Think of it like a picture frame:

- Padding = the white mat inside the frame
- Margin = the wall space around the frame

Check for Understanding

You want more space between two `<section>` elements. Which property should you change?

- A) `padding` — it controls spacing between elements
- B) `border` — making it thicker adds space
- C) `margin` — it controls space outside the element
- D) `font-size` — bigger text makes sections farther apart

Flexbox

Flexbox lets you **arrange items in a row or column**.

```
#connect ul {  
  display: flex;  
  gap: 16px;  
  list-style: none;  
  padding: 0;  
}
```

Before: links stacked in a vertical list.

After: links sit side by side in a row.

Key properties:

- `display: flex` — turns on Flexbox
- `gap` — space between flex items
- `justify-content` — aligns items left/center/right

Flexbox for the Experience Section

The assignment asks you to display experience items in columns:

```
#experience ul {  
  display: flex;  
  flex-wrap: wrap;  
  gap: 20px;  
  list-style: none;  
  padding: 0;  
}  
  
#experience li {  
  flex: 1 1 200px;  
  padding: 12px;  
  border: 1px solid #ccc;  
}
```

Each `<li>` becomes a styled card that wraps to the next row when the screen is narrow.

Navigation with `<nav>`

A `<nav>` element groups your navigation links.

Internal links use `#` followed by the section's `id`:

```
<nav>
  <a href="#about">About</a>
  <a href="#experience">Experience</a>
  <a href="#skills">Skills</a>
  <a href="#projects">Projects</a>
  <a href="#connect">Connect</a>
</nav>
```

Clicking a link **scrolls the page** to that section.

The section must have a matching `id` attribute for the link to work.

Check for Understanding

Which CSS property is required to make a Flexbox layout?

- A) `flex-direction: row`
- B) `display: flex`
- C) `gap: 20px`
- D) `justify-content: center`

Apply

Core Activity: Style a Page Together

We will build CSS step by step — **you tell me what to type!**

Open your portfolio project in VS Code.

Make sure your `css/index.css` file is linked in `index.html`.

Make one small change, save, and refresh before moving to the next step.

Step 1 — Body Styles

Let's set a foundation for the whole page:

```
body {  
  background-color: #f5f5f5;  
  color: #333;  
  font-family: 'Segoe UI', Arial, sans-serif;  
  margin: 0;  
  padding: 20px;  
}
```

Save and refresh. What changed on the page?

Step 2 — Style the Headings

```
h1 {  
  font-size: 2.5em;  
  text-align: center;  
}  
  
h2 {  
  color: #12284C;  
  border-bottom: 2px solid #FF5C35;  
  padding-bottom: 4px;  
}
```

Ask the group: What does `border-bottom` do to an `h2`?

Step 3 — Add Spacing Between Sections

```
section {  
  padding: 20px;  
  margin-bottom: 30px;  
}
```

Debugging tip — add a temporary border to see the boxes:

```
section {  
  border: 2px solid red;  
}
```

Remove the red border once you understand the layout.

Step 4 — Flexbox for the Connect Section

```
#connect ul {  
  display: flex;  
  gap: 20px;  
  list-style: none;  
  padding: 0;  
}
```

Before: links stacked vertically.

After: links arranged in a row.

Zoom chat: What does `list-style: none` remove from the list?

Step 5 — Add the Navigation Bar

Add a `<nav>` block near the top of your `index.html`:

```
<nav>
  <a href="#about">About</a>
  <a href="#experience">Experience</a>
  <a href="#skills">Skills</a>
  <a href="#projects">Projects</a>
  <a href="#connect">Connect</a>
</nav>
```

Step 5 - Continued

Style it in `index.css`:

```
nav {  
  display: flex;  
  gap: 16px;  
  padding: 12px 20px;  
  background-color: #12284C;  
}  
  
nav a { color: #FFFFFF; text-decoration: none; }
```

Extension Activity

Only if students are ready and time allows.

Extension — Sticky Navigation

Make the navigation bar stay at the top of the screen as the user scrolls:

```
nav {  
  position: sticky;  
  top: 0;  
  z-index: 100;  
}
```

- `position: sticky` — the element scrolls normally until it hits the top, then it stays
- `top: 0` — it sticks at the very top edge
- `z-index: 100` — keeps it on top of other content

Test it: add enough content to your page that it scrolls, then scroll down.

Assignment Preview

This week's assignment has two parts:

Part 1 — CSS Styling

- Link an external stylesheet
- Change background color, text color, and font
- Add padding and margin between sections
- Style headings, your name, list items, and links

Part 2 — Layout and Navigation

- Add a `<nav>` bar with internal links
- Add **Skills** and **Projects** sections (empty — filled in later)
- Use Flexbox for the Experience and Connect sections

Tips

1. **Make one change at a time** — save and refresh before adding the next rule
2. **Use DevTools to experiment** — right-click any element → Inspect → try CSS live
3. **Colorful borders are your friend** — add `border: 2px solid red` to see any box
4. **The path in `href` must be exact** — `css/index.css`, not `index.css`
5. **Stuck on a property?** Ask an AI for hints, not answers:

"Explain what `flex-wrap` does and give me one example."

Wrap-Up

Zoom chat: Rate your confidence from 1 to 5

How comfortable do you feel writing CSS rules and using Flexbox?

Before next session:

- Complete the **lesson materials** (Odin Project or Scrimba)
- Style your portfolio with `index.css`
- Add the `<nav>`, **Skills**, and **Projects** sections
- Post questions in the **Slack** `#discussion` channel anytime

Resources

Resource	What it's for
Odin Project — Intro to CSS	Selectors, properties, the cascade
Odin Project — The Box Model	Padding, margin, border
Odin Project — Intro to Flexbox	Flex containers and items
Scrimba — HTML & CSS Crash Course	Video walkthroughs
CSS-Tricks: A Guide to Flexbox	Visual flexbox reference
MDN CSS Basics	Property lookup and docs

Next week: The DOM API — using JavaScript to make your page interactive!

Great work today!

Your portfolio is starting to come alive.

Questions? Reach out on Slack anytime.